BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 8

Roll no: 51 PRN No: 12411033 Name of Student: Manjiri Kulkarni

Subject Name: Deep Learning

Problem Statement:

Implement pretrained BERT model for sentiment analysis or text classfication on a sample dataset

Algorithm:

1. Start the program.
2. Import the required libraries and load the AG News dataset.
3. Select a subset of training and testing data.
4. Load the pretrained DistilBERT tokenizer and model.
5. Tokenize the news text and assign four categories: World, Sports, Business, and Sci/Tech.
6. Configure the model using the Adam optimizer settings and other training parameters.
7. Train the model for 4 epochs using the training dataset.
8. Save the trained model and tokenizer.
9. Evaluate the model using accuracy, precision, recall, and F1-score.
10. Generate a classification report and confusion matrix.
11. Predict categories for new news articles and display the results.
12. Stop the program.

Code:

```
import numpy as np
import torch
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    Trainer,
    TrainingArguments
)
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix
)
import matplotlib.pyplot as plt
import seaborn as sns
print("Device:", "cuda" if torch.cuda.is_available() else "cpu")
# Load AG News dataset
dataset = load_dataset("fancyzhx/ag_news")
# Select a smaller dataset for our experiment
train_ds = dataset["train"].shuffle(seed=42).select(range(5000))
test_ds = dataset["test"].shuffle(seed=42).select(range(1000))
print("Training examples:", len(train_ds))
print("Testing examples :", len(test_ds))
print("\nExample news:")
print(train_ds[0]["text"])
print("\nLabel:", train_ds[0]["label"])
# Load DistilBERT
model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
# Convert text into tokens
def tokenize(data):
    return tokenizer(
        data["text"],
        truncation=True,
        padding="max_length",
        max_length=128
    )
train_ds = train_ds.map(tokenize, batched=True)
test_ds = test_ds.map(tokenize, batched=True)
print("Tokenization completed.")
# Load DistilBERT for 4-class classification
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=4
)
print("Model loaded.")
# Calculate accuracy, precision, recall and F1
def metrics(eval_pred):
    logits, labels = eval_pred
```

```
    predictions = np.argmax(logits, axis=1)
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels,
        predictions,
        average="weighted"
    )
    accuracy = accuracy_score(
        labels,
        predictions
    )
    return {
        "accuracy":  accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
# Training settings
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=4,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=16,
    learning_rate=2e-5,
    eval_strategy="epoch",
    save_strategy="epoch",
    save_total_limit=1,
    report_to="none"
)
# Create Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=test_ds,
    processing_class=tokenizer,
    compute_metrics=metrics
)
# Train DistilBERT
trainer.train()
# Save the trained model
trainer.save_model("./final_model")
tokenizer.save_pretrained("./final_model")
print("Model saved successfully.")
import os
print(os.listdir("./final_model"))
# Evaluate the model
results = trainer.evaluate()
print("Evaluation Results")
print("-" * 40)
for key, value in results.items():
    if isinstance(value, float):
```

```
        print(f"{key}: {value:.4f}")
    else:
        print(f"{key}: {value}")
# Get predictions
pred = trainer.predict(test_ds)
predictions = np.argmax(
    pred.predictions,
    axis=1
)
true_labels = pred.label_ids
# Display classification report
print(
    classification_report(
        true_labels,
        predictions,
        target_names=[
            "WORLD",
            "SPORTS",
            "BUSINESS",
            "SCI/TECH"
        ],
        digits=4
    )
)
# Create confusion matrix
cm = confusion_matrix(
    true_labels,
    predictions
)
print(cm)
# Plot confusion matrix
plt.figure(figsize=(7, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=[
        "WORLD",
        "SPORTS",
        "BUSINESS",
        "SCI/TECH"
    ],
    yticklabels=[
        "WORLD",
        "SPORTS",
        "BUSINESS",
        "SCI/TECH"
    ]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.title("DistilBERT - AG News Confusion Matrix")
plt.show()
# Predict the category of new news
def predict_news(text):
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    )
    inputs = {
        key: value.to(model.device)
        for key, value in inputs.items()
    }
    with torch.no_grad():
        output = model(**inputs)
    prediction = torch.argmax(
        output.logits,
        dim=1
    ).item()
    categories = [
        "WORLD",
        "SPORTS",
        "BUSINESS",
        "SCI/TECH"
    ]
    return categories[prediction]
# Test the model with new news
test_news = [
    "India won the cricket match against Australia.",
    "The company announced a new business deal.",
    "Scientists developed a new artificial intelligence system.",
    "World leaders met to discuss international relations."
]
for news in test_news:
    print("News:", news)
    print("Category:", predict_news(news))
    print()
```

Output:

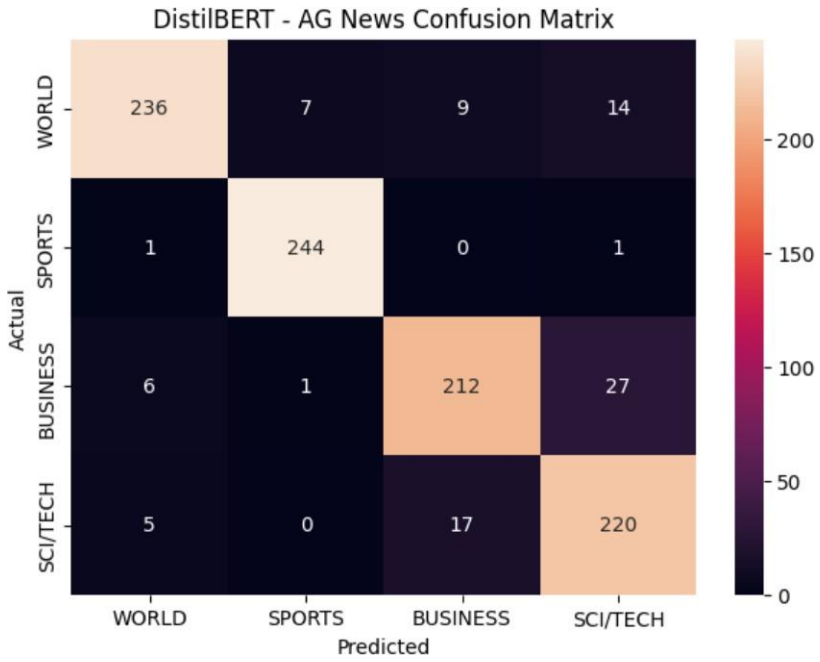
Training metrics:

Training Loss	Validation Loss	Epoch	Accuracy	Precision	Recall	F1
0.125727	0.404802	4	0.912000	0.913652	0.912000	0.912099

```
Evaluation Results
----------------------------------------
eval_loss: 0.4048
eval_accuracy: 0.9120
eval_precision: 0.9137
eval_recall: 0.9120
eval_f1: 0.9121
```

Evaluation metrics:

	precision	recall	f1-score	support
WORLD	0.9516	0.8872	0.9183	266
SPORTS	0.9683	0.9919	0.9799	246
BUSINESS	0.8908	0.8618	0.8760	246
SCI/TECH	0.8397	0.9091	0.8730	242
accuracy			0.9120	1000
macro avg	0.9126	0.9125	0.9118	1000
weighted avg	0.9137	0.9120	0.9121	1000



Test headlines:

```
News: India won the cricket match against Australia.
Category: SPORTS

News: The company announced a new business deal.
Category: BUSINESS

News: Scientists developed a new artificial intelligence system.
Category: SCI/TECH

News: World leaders met to discuss international relations.
Category: WORLD
```